

# Bài 3:

## Kế thừa-Inheritance

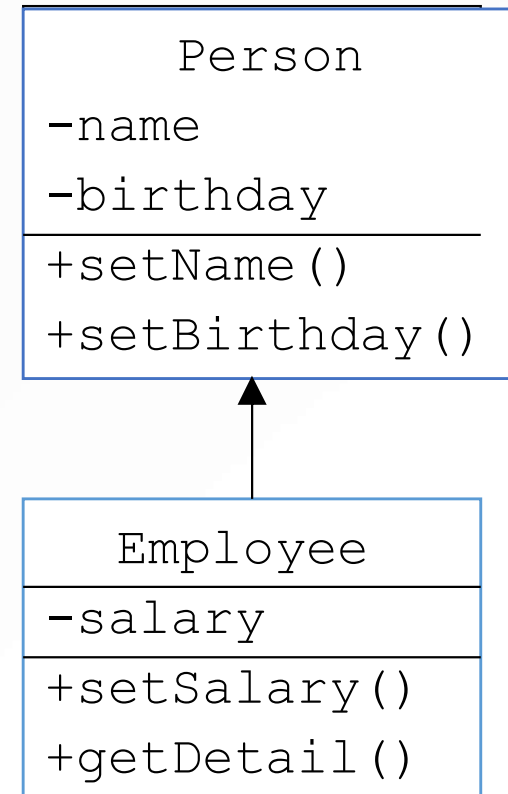
# Khái niệm kế thừa

---

- Thừa hưởng lại các thuộc tính và phương thức đã có từ lớp cha (super class)
- Chi tiết hóa cho phù hợp với mục đích sử dụng tại lớp mới
  - + Thêm các thuộc tính mới
  - + Thêm hoặc hiệu chỉnh các phương thức đã có theo “mong đợi” của lớp mới

# Khái niệm kế thừa

- Java cung cấp cơ chế phân cấp có thứ bậc giữa các class, gọi là cây thừa kế
- Class bậc trên gọi là lớp cha (super class), các class bậc dưới được gọi là các lớp con (sub class)
- Trong Java, 1 lớp chỉ có thể có 1 lớp cha duy nhất (đơn thừa kế), điều này thuận với tự nhiên, tránh bị xung đột.



# Từ khóa **extends**

---

- Extends được sử dụng để chỉ ra lớp cha (super class) muốn thừa kế.

```
modifier class SubClassName extends SupperClassName {
```

```
    Các thuộc tính của lớp con
```

```
    Các phương thức lớp con
```

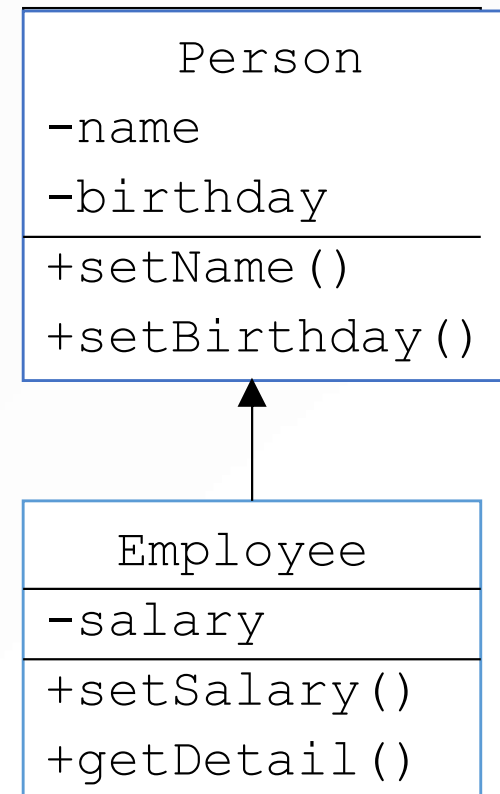
```
}
```

# Ví dụ

- Person: là lớp mô tả thông tin cơ bản của mỗi người (name, birthday)
- Employee: cũng là con người nên có tên, ngày sinh
- **Ngoài ra:** Employee là nhân viên thì có thêm thông tin về mức lương

Employee sẽ kế thừa lớp Person để thừa hưởng thuộc tính **name, birthday** của Person  
Employee sẽ thêm thuộc tính **salary** cho riêng mình

Số thuộc tính của Employee:  
**name, birthday, salary**



# Ví dụ

```
5 public class Person {  
6  
7     public String name;  
8     public Date birthday;  
9  
10    public Person(){  
11  
12    }  
13    public String getName() {  
14        return name;  
15    }  
16    public void setName(String name) {  
17        this.name = name;  
18    }  
19    public Date getBirthday() {  
20        return birthday;  
21    }  
22    public void setBirthday(Date birthday) {  
23        this.birthday = birthday;  
24    }  
25 }
```

```
3 public class Employee extends Person{  
4  
5     public double salary;  
6  
7     public double getSalary() {  
8         return salary;  
9     }  
10  
11    public void setSalary(double salary) {  
12        this.salary = salary;  
13    }  
14 }
```

# Phạm vi kế thừa

---

- **public**: thuộc tính/phương thức của lớp cha cho phép tất cả các lớp (con, cháu,...) kế thừa.
- **private**: thuộc tính/phương thức của lớp cha không cho lớp con kế thừa

# Từ khóa **this** , **super**

**this**: tham chiếu đến đối tượng hiện tại

**super**: tham chiếu đến “lớp cha”

```
3 public class Employee extends Person{
4
5     public double salary;
6
7     public void ShowInfo(){
8
9         System.out.println("Ten nhan vien:" + super.getName());
10        System.out.println("Ngày sinh:" + super.getBirthDay().toString());
11        System.out.println("Muc luong:" + this.salary);
12    }
13
```



# Overwrite

---

- Overriding là kỹ thuật cho phép định nghĩa lại một phương thức đã có ở lớp cha – super class, để sử dụng cho phù hợp với thiết kế của lớp con – sub class

# Overwrite

---

- Lớp con overwrite phương thức của lớp cha làm cho phương thức của lớp cha bị mất hiệu lực đối với lớp con.
- Mục đích của overwrite là chỉnh sửa lại phương thức được thừa thừa kế cho phù hợp với mục đích sử dụng của lớp con (không thay đổi tên phương thức)

# Ví dụ

```
5 public class Person {
6
7     public String name;
8     public Date birthday;
9
10
11     protected void ShowInfo(){
12         System.out.println("*****THONG TIN*****");
13         System.out.println("Name: " + name);
14         System.out.println("Birthday: " + birthday.toString());
15     }
16 }
```

```
7 public class Employee extends Person{
8
9     public double salary;
10
11     public void ShowInfo(){
12         System.out.println("*****THONG TIN NHAN VIEN*****");
13         System.out.println("Ten nhan vien: " + super.getName());
14         System.out.println("Ngay sinh: " + super.getBirthday().toString());
15         System.out.println("Muc luong: " + this.salary);
16     }
17 }
```

# Overwrite

- Trường hợp lớp con muốn dùng phương thức đã overwrite của lớp cha thì dùng từ khóa **super** để truy cập vào phương thức của lớp cha.

```
7 public class Employee extends Person{
8
9     public double salary;
10
11     public void ShowInfo(){
12
13         super.ShowInfo();
14
15         System.out.println("Muc luong: " + this.salary);
16     }
17 }
```